Network Robustness: Percolation Theory and Attack Tolerance

Part 1: Introduction to Network Robustness and Percolation Theory

1.1. Background

Many real-world systems—such as the Internet, power grids, and social networks—rely on their connectivity to function. **Robustness** refers to a network's ability to maintain functionality when nodes or edges are removed. **Percolation theory** provides a mathematical framework to study the resilience of networks under random failures or targeted attacks.

1.2. Key Concepts

- **Percolation Threshold:** The critical fraction of nodes or edges that must be present for a giant connected component to exist.
- **Giant Component:** The largest connected cluster in the network.
- **Random Failure vs. Targeted Attack:**
    - *Random Failure:* Nodes are removed randomly.
    - *Targeted Attack:* Nodes are removed in order of decreasing degree.

Part 2: Robustness of Scale-Free Networks

2.1. Theoretical Background

Scale-free networks (with degree distribution $P(k) \sim k^{-\gamma}$)are known to be **robust to random failures** but **vulnerable to targeted attacks**. This is due to the presence of a few highly connected hubs.

2.2. Simulation: Random Failure vs. Targeted Attack

Write a Python function `simulate_robustness(graph, attack_type='random')` that:

1. Takes a network and an attack type (random or targeted).
2. Removes nodes one by one (without replacement).
3. At each step, computes the size of the largest connected component.
4. Returns a list of relative giant component sizes vs. fraction of nodes removed.

Instructions:

- Use a BA model graph with $N=5000$, $m=2$ for testing.
- Compare the results of random failure and targeted attack.
- Plot the relative size of the giant component vs. fraction of nodes removed.

Part 3: Cascading Failures

3.1. Model Definition

Cascading failures occur when the failure of one node overloads others, leading to a chain reaction. A simple model:

- Each node has a load L_i (e.g., degree or initial betweenness centrality).
- Each node has a capacity $C_i = (1+\alpha)L_i$, where α is a tolerance parameter.
- If a node fails, its load is redistributed among others using a **clearly defined mechanism**. Options include:.
    - Equal redistribution to all neighbors
    - Redistribution proportional to neighbor capacity
    - Flow-based redistribution (requires recomputing shortest paths)

- If any node's load exceeds its capacity, it also fails.

3.2. Implementation

Write a function `simulate_cascading_failures(graph, alpha=0.2, initial_failure=None, redistribution_method='equal')` that:

1. Computes initial load for all nodes (using degree for efficiency, or betweenness for small networks).
2. Fails an initial node (or set of nodes).
3. **Redistributes load** according to the specified method and checks for new failures iteratively.

4. **Recomputes loads if necessary** (for flow-based methods).
5. Stops when no new failures occur.
6. Returns the total number of failed nodes.

Experiment:

- Run the simulation on a BA network and a real-world network.
- Vary α and observe the effect on cascade size.

Part 4: Building Robust Networks

4.1. Strategies for Robustness

- **Intentional Design:** Add edges to increase connectivity.
- **Protecting Hubs:** In scale-free networks, protecting high-degree nodes can significantly improve robustness.
- **Modularity and Redundancy:** Design networks with redundant paths and modular structure.

4.2. Exercise: Improving Robustness

Take a real-world network and:

1. Measure its robustness under targeted attack, using the robustness.metric: $R = \frac{1}{N}\sum_{q=0}^{1} s(q)$ where s(q) is the relative size of the largest component after removing fraction of q nodes
2. Propose and test a strategy to improve its robustness:
    - Add k edges between highest-degree nodes
    - Add k edges between lowest-degree nodes
    - Add k edges using max-min degree connection
    - Protect top p% of high-degree nodes from attack

3. Quantify the improvement using the area under the robustness curve.
    - Area Under the robustness Curve (AUC)
    - Percentage improvement in robustness metric R
    - Change in percolation threshold